

Video Transcript

Video 10: Security Overview: Kerberos (11:43)

Two of the building blocks for security are authentication and authorisation. Authentication addresses the problem of who are you and authorisation says and what are you allowed to do. So, let's take a look. We're going to use Kerberos as our example. This was developed in MIT and here's the setting that it applies to. So, we have a number of users and they need to be able to use multiple services to log into those services and for example, if they're using e-mail, they may need to do it multiple times a day. Now, we don't want to have them log in and give their password every time they use it.

What we'd like to do is the first time that day that the user logs in to the e-mail server, we'd like from then on that to be handled behind the scenes and to be automatic and also we'd like to make sure that we have security and so here we have an attacker, and we assume that the attacker can monitor any of the traffic, any of the messages that are going between these parties. Now, to get over the N^2 problem that we saw here, where if each of these services had to store the passwords of all the users, we'd run into an N^2 problem in the sense that if there were N users and services, then we'd have N^2 storage of passwords.

So, let's take a look at the assumptions. We don't trust the network to be secure. We assume attackers can both see and copy and replay any message that goes between the parties. We'd like to have single sign on so users don't have to enter passwords multiple times. The passwords should never be passed in clear text on the network for obvious reasons, and the passwords should never be stored on the client's machine. So, let's see how we get around that. Obviously, the Kerberos server here gets over the N^2 problem. Now, each of the parties trusts Kerberos, the users trust Kerberos, and the services trust Kerberos.

Kerberos needs to store. You'd think they would need to store the passwords, but actually we'll see the they don't need to store the password, they need to know the user's password. They are going to use that password to generate a key, so, a secret key for that user. Let's see how they might do that. So, for example, they could take the hash of the username and the password and create that key and then that key could be used to encrypt messages. So, we'll treat that key as a

symmetric encryption key. So, we can apply it to 'Hello World,' and we'd get our encrypted text and then we could apply the key again and decrypt and get back to 'Hello World,' and that's the way it's going to be used in Kerberos.

As strange as it may seem, the passwords are never stored. So, what we see here is that the user knows their password. So, Alice knows what her password is. She doesn't need to store it anywhere on the machine. She'll just keep it in her head and over here, the username and the password are hashed together to create a key that can be used in encryption. So, Alice's key, I'm calling Akey. Now, the services also need a secret key, and so they have one and here we're going to be dealing with the e-mail server, and I've called it Vkey.

So, that's our set up. Let's take a look now at how we might go about providing access to a service. So, the first thing that Alice does that day is she sends a message to the Kerberos server requesting a ticket granting ticket. So, she sends her username and the ticket granting ticket request, and that's in plain text. Now, Kerberos looks up Alice and retrieves his secret key and Kerberos, assuming she's Alice, is found untrusted. Kerberos issues a ticket granting key which is encrypted using Alice's secret key. So, now she gets back an encrypted message and she can decrypt it. So, Alice can get the ticket granting ticket.

And so, that ticket granting key is stored in Alice's tray. We call it for the whole day and expires after say eight hours. Now, the ticket granting ticket is used rather than the password. So, now Alice makes a request for access to the e-mail server and she passes to Kerberos the ticket granting ticket. Kerberos recognises this and if Alice has the right permissions, Kerberos issues an e-mail token and that's encrypted with the services secret key. Here I've called it Vkey. So, this is what Alice gets back. the Vkey encrypted username, and service name, and that's the token.

Okay, so now Alice sends this e-mail token to the e-mail server and the server decrypts it with it's Vkey so it knows the message is approved because only Kerberos could have encrypted with it's secret key and that Alice should be given access to the e-mail server. Now, the problem with this is that suppose the attacker is watching and captures this e-mail token and just copies it and spins up it's own machine, pretends to be Alice and sends the token to the e-mail server. So, now the attacker will get back Alice's e-mail. So, we need to think of a slightly different system.

So, we might include the IP address of Alice for example, in this e-ticket, but even that the attacker can fake. It's quite easy to spin up a machine and send messages with. In the TCP packet, you can have a fake address for the origin. So, that doesn't actually solve the problem. We have

to think a bit more deeply about this. So, let's go back to the beginning now and Alice is now going to request a ticket granting ticket.

This time the Kerberos server creates a session key, we'll call it session key one, and message A comes back with that session key encrypted with Alice's key. So, Alice can read that and get the session key one token and she gets back the ticket granting ticket and that happens to be something that's encrypted with the master key of the Kerberos server and contains her username, address, the lifespan of the ticket and the session key. Now, Alice can request access to the e-mail server. This time she sends two messages to the Kerberos server. She sends the ticket, granting ticket and the name of the service that she wants to access and she also sends back her username and time stamp encrypted with the session one key that she's already obtained. Now, Kerberos recognizes the ticket granting ticket, and if everything's okay, issues the e-mail ticket encrypted with the e-mail services key, secret key, the Vkey.

So, that gets sent back. Let's call that the e-ticket and the Kerberos server creates another session key, this time to refer to the session that's going to occur between Alice and the e-mail servers, and encrypts that with the session key one, and those get sent back as messages ENF to Alice. She has session key one, so she can decrypt this message. She doesn't have the Vkey, she can't decrypt that. So, now what does Alice do? Well, she sends now with the session key two used as the encryptor username and timestamp and she includes the e-ticket key.

So, these are sent to the services. The service, the e-mail service has Vkey, so, can decrypt this message and can get the session key two key, and can therefore now decrypt this message F and can see the username and timestamp that Alice has sent her. Now, for an attacker, the attacker wouldn't be able to get this session key too, so, wouldn't be able to create this message for the services. So, that solves this problem. Here's a confirmation message sent back to Alice with the session key two and the time stamp. And so, she can be assured now that she can get her e-mail. So, that's basically how Kerberos works. So, here I've just set out the steps and you can go through those, but it's a very slick system. This is probably one of the best standards used by Microsoft and is very secure. Okay, that's all for now.